

Presented by Ayush Indapure

Student ID : 224880003

SOFTWARE QUALITY AND TESTING

BOUNDARY VALUE ANALYSIS & EQUIVALENCE CLASS TESTING

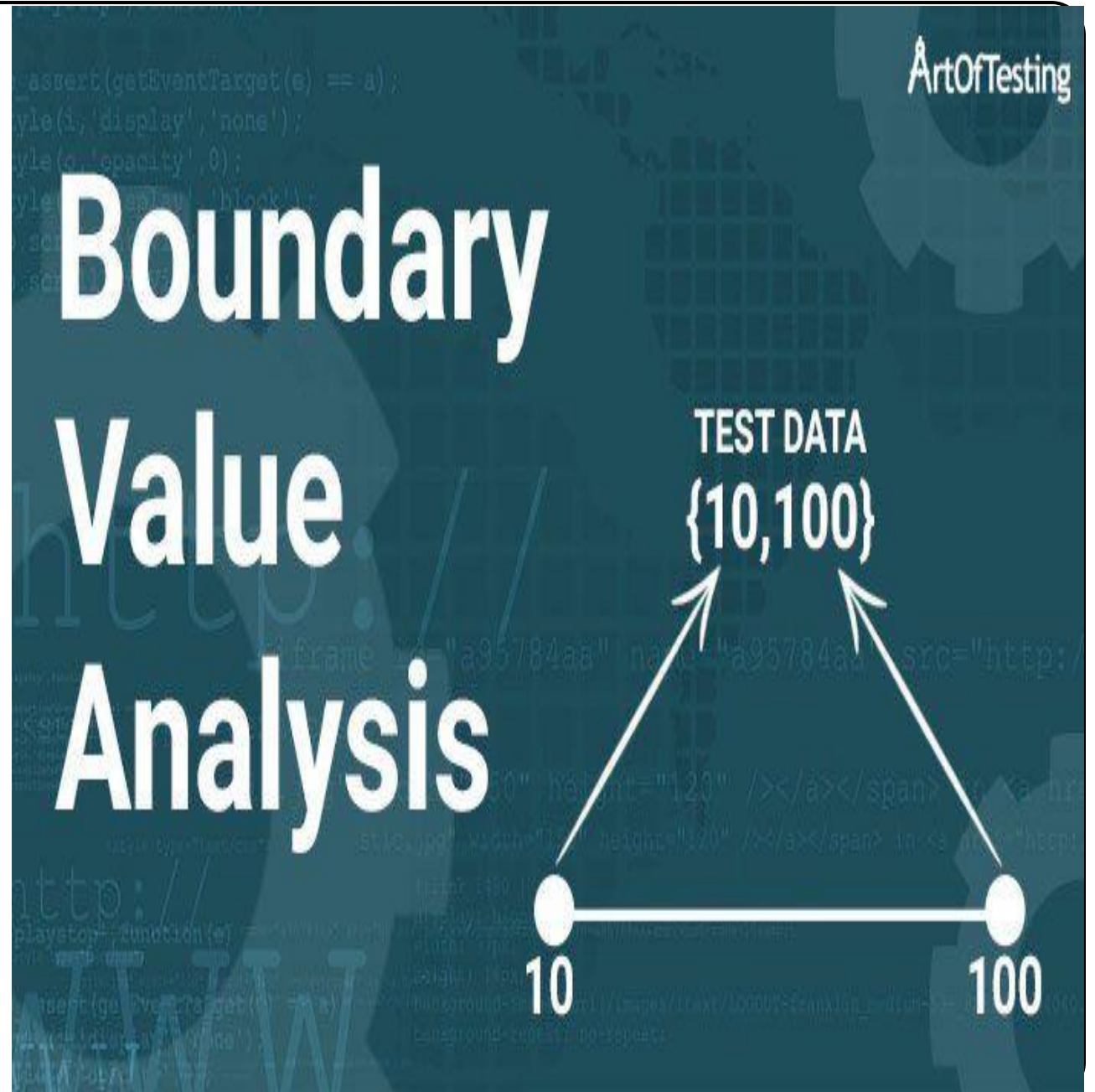
This week I learned about two important testing strategies

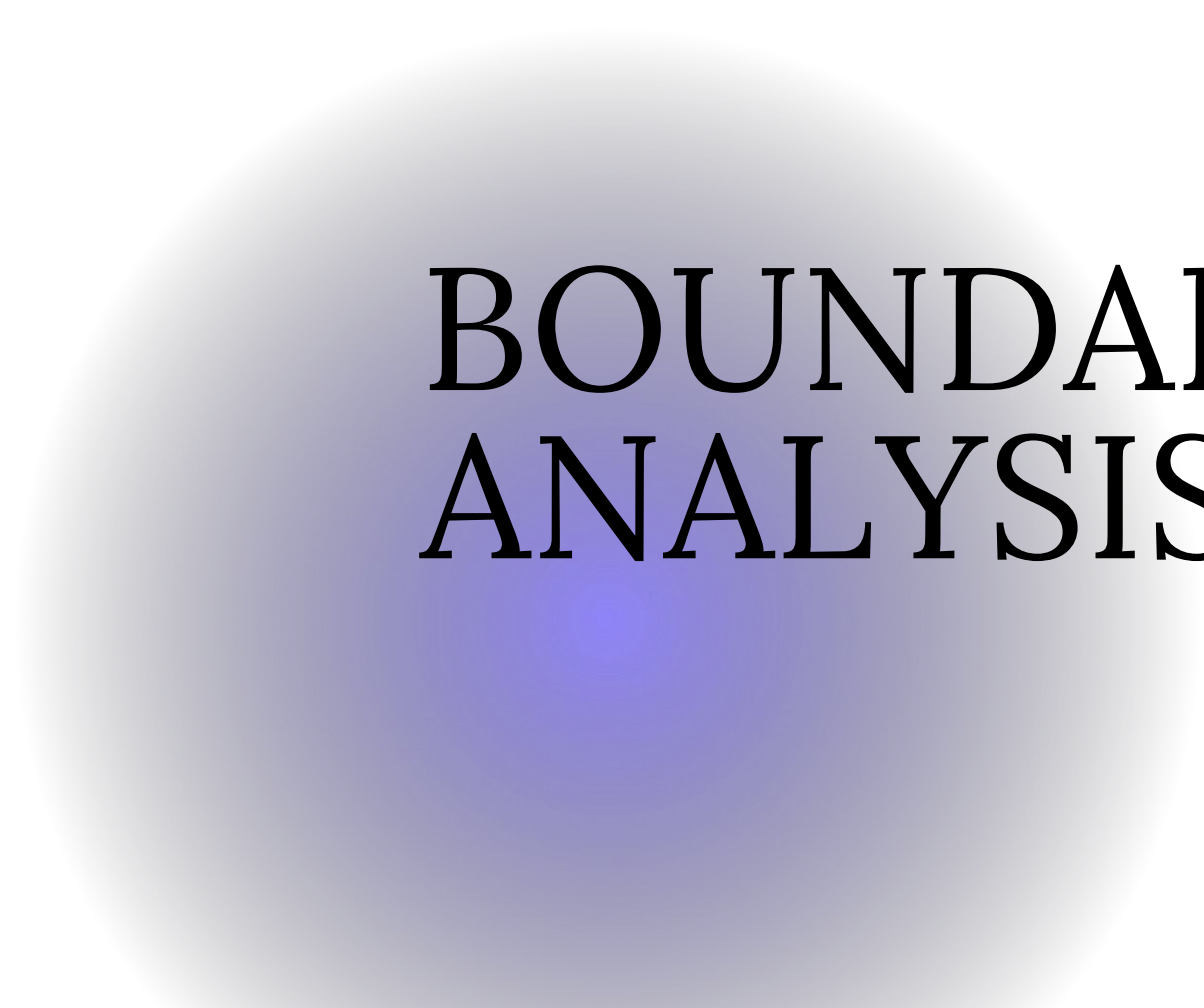
- Boundary Value Analysis (BVA)
- Equivalence Class Testing (ECT)

Both techniques help testers design **efficient test cases** without testing every possible input.

Instead of testing thousands of inputs, we focus on **critical input ranges and edge cases**.

These strategies are commonly used in real-world software systems where inputs have **specific limits or constraints**.





BOUNDARY VALUE ANALYSIS

BVA

(Boundary Value Analysis)

**BOUNDARY VALUE
ANALYSIS FOCUSES
ON TESTING VALUES
AT THE EDGES OF
INPUT RANGES.**

BOUNDARY VALUE ANALYSIS

Boundary Value Analysis (BVA) is a software testing technique that basically focuses on testing input values at the edges of defined ranges, as defects are most likely to occur at these boundaries rather than in the middle. .

This black-box testing method is based on the observation that errors frequently happen when values are just below, at, or just above the valid limits of an equivalence partition. Testers design test cases to cover the minimum, just above minimum, nominal, just below maximum, and maximum values of a range, often testing both valid and invalid partitions.

Black Box Testing Techniques

2. Boundary Value Analysis



REAL EXAMPLE I'VE
SEEN (DSA CONTEXT)

EXAMPLE THROUGH DATA STRUCTURES AND ALGORITHMS

While practicing **Data Structures and Algorithms**, I noticed boundary problems often appear in array indexing.

Example:

```
int[] arr = new int[10];
```

Valid indexes are:

0 – 9

If a program tries to access:

```
arr[10]
```

it causes an **ArrayIndexOutOfBoundsException**.

So the boundary values are:

0, 1, 8, 9

Testing these helps ensure the algorithm handles edge cases correctly.

This is very similar to Boundary Value Analysis used in software testing.

EQUIVALENCE CLASS TESTING(ECT)

Equivalence Class Testing divides input data into **groups (classes)** where the program should behave the same. Instead of testing every possible input, we select **one representative value from each class**.

This black-box testing method is based on the observation that errors frequently happen when values are just below, at, or just above the valid limits of an equivalence partition. Testers design test cases to cover the minimum, just above minimum, nominal, just below maximum, and maximum values of a range, often testing both valid and invalid partitions.

Black Box Testing Techniques

1. Equivalence Class Testing



EXAMPLE THROUGH DATA STRUCTURES AND ALGORITHMS

A common real-world example is **password validation**.

Suppose a system requires:

Password length: 8 – 16 characters

Equivalence classes would be:

Valid class

8–16 characters

Invalid classes

< 8 characters

> 16 characters

Instead of testing every possible length, testers choose **representative values** like:

7, 8, 12, 16, 17

HOW THESE METHODS ARE USED IN REAL SOFTWARE

Both testing techniques are widely used in real systems:

Examples include:

- Banking systems (transaction limits)
 - E-commerce websites (quantity limits)
 - Input validation in web forms
 - Date calculators and scheduling systems
 - API request parameter validation

By focusing on boundaries and input groups, testers can **reduce the number of tests while still achieving high coverage.**

REFLECTION

Both testing techniques are widely used in real systems:

Examples include:

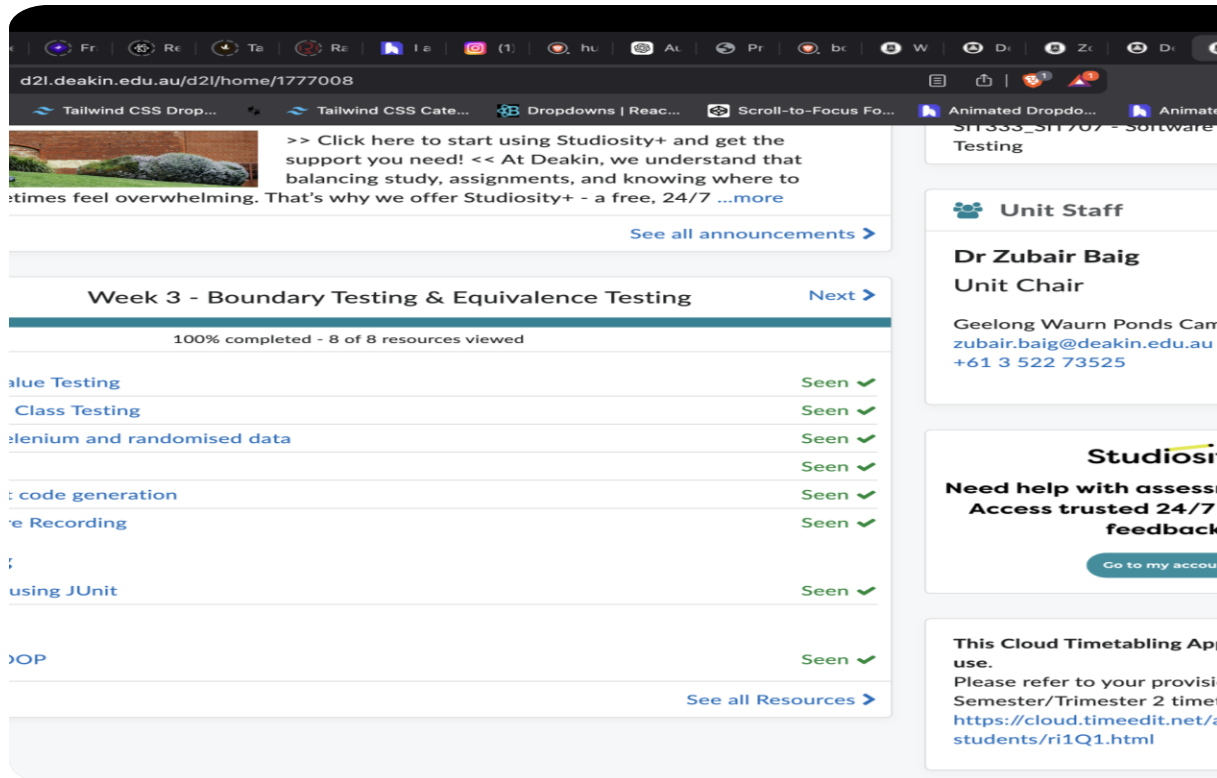
- Banking systems (transaction limits)
 - E-commerce websites (quantity limits)
 - Input validation in web forms
 - Date calculators and scheduling systems
 - API request parameter validation

By focusing on boundaries and input groups, testers can **reduce the number of tests while still achieving high coverage.**

EVIDENCE OF LEARNING ACTIVITY

During the active learning session I explored:

- Boundary testing for a **date calculator program**
 - Using JUnit test cases to validate date boundaries
 - Testing edge cases such as:
 - Start of month
 - End of month
 - Leap year conditions



Thank You!

SIT707 – Software Quality and Testing

Student Name : Ayush Indapure

Student ID : 224880003

Introduction :

In this task, I basically implemented unit testing for the dateUtil class using the JUnit testing framework.

The purpose of this task was to verify that the date calculation functions work correctly for different scenarios such as normal days, month boundaries, and leap years.

Unit testing helps ensure that individual components of a program behave as expected. By creating automated test cases, developers can quickly verify whether the program still works correctly after making changes.

In this task, I created multiple test cases to check both the **next date** and **previous date** functionality of the program.

Part A – Test Case Design

While designing the test cases, I tried to include scenarios that represent real-world situations where date calculations might cause errors.

For example, firstly changed the name to Ayush Indapure and student ID to 224880003 when the date is at the end of a month, the next date should move to the first day of the next month. Similarly, leap years must be handled correctly because February has 29 days instead of 28.

Some of the important scenarios tested in this assignment include:

Normal Date Increment

A standard date such as **15 March 2023** should correctly increment to **16 March 2023**.

Month Boundary

When the date reaches the last day of a month such as **31 January**, the next date should become **1 February**.

Leap Year Case

In leap years, February has 29 days. Therefore a date like **28 February 2024** should increment to **29 February 2024**.

Previous Date Cases

The program should also correctly calculate the previous date. For example:

- **1 March 2023 → 28 February 2023**
- **1 January 2023 → 31 December 2022**

These cases help verify that the program correctly handles transitions between months and years.

Part B – JUnit Implementation

JUnit was initially used to automate the testing process for the DateUtil class. The tests were written inside the DateUtilTest case.

Each test method uses the @test notation and checks whether the expected output matches the actual output returned by the program.

For example, when testing the next date functionality, the program creates a date object and verifies whether the nextDate() method returns the correct value.

The assertEquals() function from JUnit was used to compare the expected result with the actual result returned by the program.

This approach allows the tests to run automatically and quickly detect any issues in the date calculation logic

Test Execution

All test cases were executed using JUnit in Visual Studio Code. After running the tests, the testing panel showed that all test cases executed successfully without any failures.

The results indicated that the program correctly handles:

- Normal date transitions
- Month boundary transitions
- Leap year scenarios
- Previous date calculations

JUnit Test Results

The JUnit test execution confirmed that all test cases passed successfully.

Example result:

Tests run: 14

Failures: 0

Errors: 0

Skipped: 0

This confirms that the **DateUtil** program behaves correctly for the tested scenarios.

.

